

Lumma Looming

Ticket Overview

Ticket Number: SOC-24117

Date/Time Reported: 2024-05-29 / 12:40:51 UTC

Assigned Analyst: Jeremiah Tindana (Contractor at MockingCyber)

Affected Endpoint: 10.0.2.10

Affected User: N/A

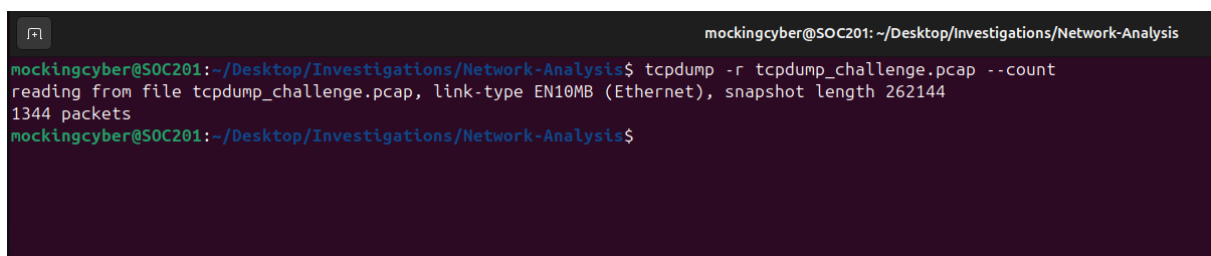
Incident Summary

On May 29, 2024 at 12:40, the SOC received an EDR alert flagging endpoint for abnormal outbound network behavior, correlating with several detections consistent with a potential info-stealer malware variant. A packet capture (tcpdump_challenge.pcap) of the host's traffic was pulled for review. MockingCyber's analyst, Jeremiah Tindana, must review the capture to identify the nature of the traffic, extract relevant indicators, and determine whether it confirms an info-stealer compromise.

NB: The sample analyzed in this report (tcpdump_challenge.pcap) is course-provided lab material from TCM Security's SOC Analyst training. "MockingCyber," the ticket number, host/user details, and timestamp are fictional, constructed for realistic case-report formatting. All investigation, IOC extraction, and analysis is my own independent work performed against the file.

Investigation

I began by getting an initial read on the capture using tcpdump's count flag.

A terminal window with a dark background. The prompt is 'mockingcyber@SOC201: ~/Desktop/Investigations/Network-Analysis'. The command entered is 'tcpdump -r tcpdump_challenge.pcap --count'. The output shows 'reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144' and '1344 packets'.

```
mockingcyber@SOC201: ~/Desktop/Investigations/Network-Analysis$ tcpdump -r tcpdump_challenge.pcap --count
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
1344 packets
mockingcyber@SOC201: ~/Desktop/Investigations/Network-Analysis$
```

Picture 1. tcpdump packet count on tcpdump_challenge.pcap

The capture contains 1,344 packets, using an Ethernet (EN10MB) link-layer type with a snapshot length of 262144. This means that packets were captured at full length, so the payload data should be recoverable.

My next step was to examine the general structure of the packets to assist with further analysis. My end goal for doing this was to examine the IPs that are present in the PCAP using the cut tool.

```
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap -c 10
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
1716994431.049874 IP 10.0.2.10.55639 > dns.google.domain: 52665+ [1au] A? meter.net. (38)
1716994431.050591 IP 10.0.2.10.34398 > dns.google.domain: 532+ [1au] AAAA? meter.net. (38)
1716994431.079304 IP dns.google.domain > 10.0.2.10.34398: 532 0/1/1 (102)
1716994431.079363 IP dns.google.domain > 10.0.2.10.55639: 52665 3/0/1 A 172.67.72.15, A 104.26.5.16, A 104.26.4.16 (86)
1716994431.081222 IP 10.0.2.10 > 172.67.72.15: ICMP echo request, id 3940, seq 1, length 64
1716994431.108514 IP 172.67.72.15 > 10.0.2.10: ICMP echo reply, id 3940, seq 1, length 64
1716994431.108809 IP 10.0.2.10.54135 > dns.google.domain: 19617+ [1au] PTR? 15.72.67.172.in-addr.arpa. (54)
1716994431.141228 IP dns.google.domain > 10.0.2.10.54135: 19617 NXDomain 0/1/1 (116)
1716994432.081541 IP 10.0.2.10 > 172.67.72.15: ICMP echo request, id 3940, seq 2, length 64
1716994432.100611 IP 172.67.72.15 > 10.0.2.10: ICMP echo reply, id 3940, seq 2, length 64
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$
```

Picture 2. First 10 packets of tcpdump_challenge.pcap with timestamps

I then filtered on TCP traffic and extracted the address field.

```
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap -n tcp -c 10 | cut -d " " -f 3
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
10.0.2.10.60032
194.108.117.16.21
10.0.2.10.60032
194.108.117.16.21
10.0.2.10.60032
194.108.117.16.21
10.0.2.10.60032
10.0.2.10.60032
194.108.117.16.21
10.0.2.10.60032
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$
```

Picture 3. TCP conversation addresses (first 10 packets)

Examining the first a few TCP packet reveals that there is a likely FTP traffic on port 21. FTP is a cleartext protocol, and info-stealers commonly use it to exfiltrate harvested credentials and files, since it requires no TLS setup. I examine the ports further to see how many commonly used ports are present in the PCAP. I did this by extracting the port from the IP field and aggregating the count of them in descending order using cut, sort and uniq functions.

```
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap -n tcp | cut -d " " -f 3 | cut -d "." -f 5 | sort | uniq -c | sort -nr
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
356 80
114 44174
56 21
23 40124
11 60032
11 47582
```

Picture 4. TCP port frequency across full capture

Port 80 dominates with 356 occurrences. This is normal because it is standard HTTP. Port 44174 appears 114 times. It is most likely a local source port rather than a destination service. Port 21 (FTP) appears 56 times. FTP traffic in this PCAP is worth a deeper dive. But before getting into the FTP data, I decided to examine the conversations more to look into IPs.

```

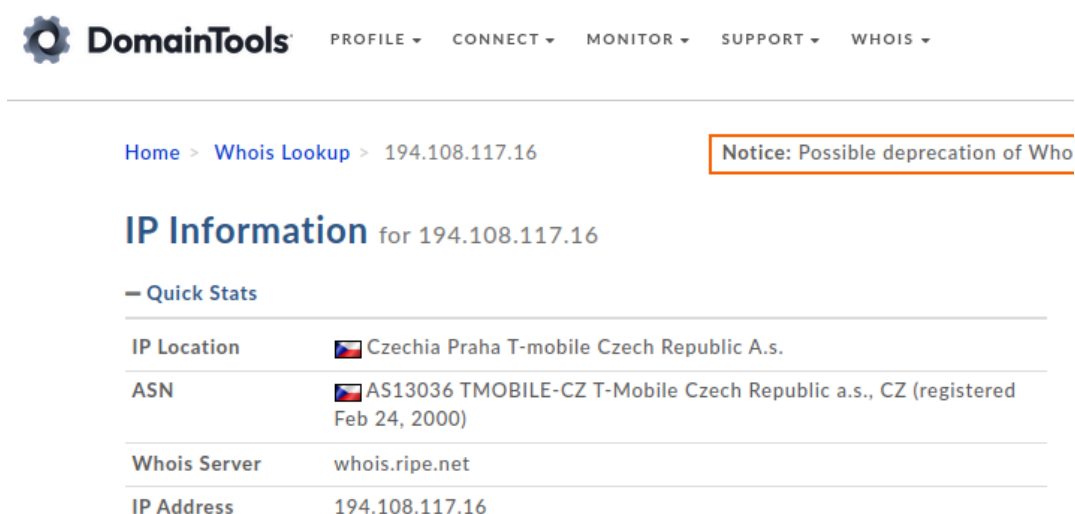
mockingcyber@50C201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap -n tcp | cut -d " " -f 3 | cut -d "." -f 1-4 | sort | uniq -c | sort -nr
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
510 10.0.2.10
113 184.84.32.184
60 194.108.117.16
18 171.161.116.100
9 23.48.37.21
9 142.251.41.78
8 151.101.1.140
6 93.184.215.14
6 23.48.37.4
5 208.82.237.241
5 208.80.154.224
5 184.26.154.94
5 18.245.96.94
5 151.101.2.187
5 151.101.2.114
5 151.101.1.224
5 151.101.1.111
5 142.251.33.174

```

Picture 5. Full source IP address frequency across TCP traffic

10.0.2.10 has 510 occurrences. This is most likely the local endpoint itself, appearing in essentially every conversation. 184.84.32.184 is the remote IP with the highest 113 packets. The FTP host 194.108.117.16 had 60 packets. This puts 184.84.32.184 in focus with the FTP packets host.

The remainder of the list is a long list of low-count connections. This pattern is typical of ordinary web browsing pulling resources from multiple third-party hosts. These can be revisited and investigated with sufficient reason for suspicion. I ran a WHOIS lookup on both IPs via DomainTools.



DomainTools

HOME > Whois Lookup > 194.108.117.16

Notice: Possible deprecation of Who

IP Information for 194.108.117.16

— Quick Stats

IP Location	Czechia Praha T-mobile Czech Republic A.s.
ASN	AS13036 TMOBILE-CZ T-Mobile Czech Republic a.s., CZ (registered Feb 24, 2000)
Whois Server	whois.ripe.net
IP Address	194.108.117.16

Picture 6. WHOIS lookup for 194.108.117.16

The address resolves to AS13036, T-Mobile Czech Republic, a mobile/consumer ISP allocation in Prague, Czech. Legitimate business FTP services are never hosted on residential/mobile carrier IP space. An FTP server sitting on a mobile ISP address is very suspicious of either a compromised residential host being used as attacker infrastructure, or the attacker's own consumer connection acting as a drop point for exfiltrated data. I ran the same WHOIS lookup against 184.84.32.184, the top talker by volume after the local machine.

 **DomainTools**

PROFILE ▾ CONNECT ▾ MONITOR ▾ SUPPORT ▾ WHOIS ▾

[Home](#) > [Whois Lookup](#) > 184.84.32.184

Notice: Possible deprecation of Who

IP Information

 for 184.84.32.184

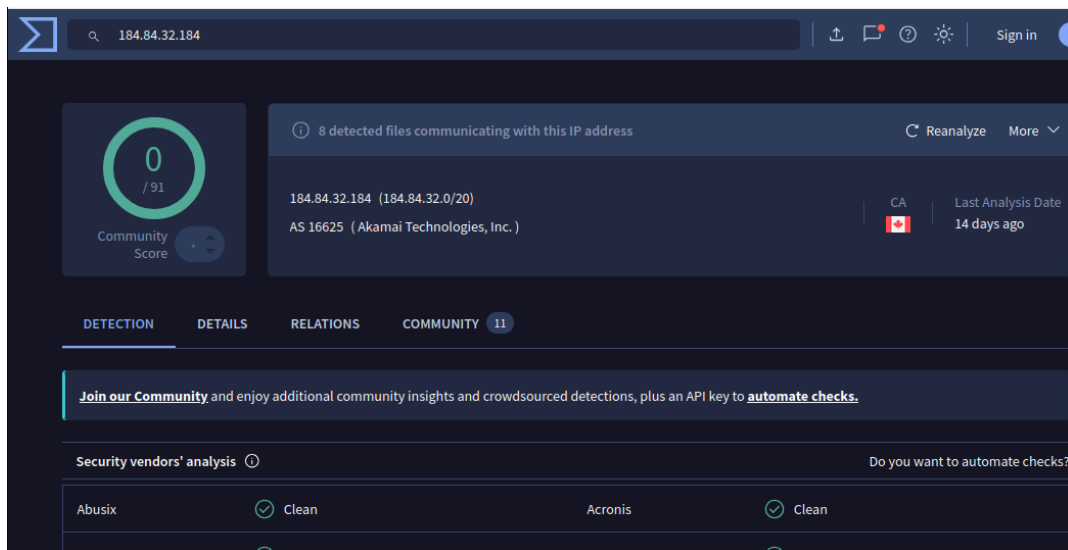
— Quick Stats

IP Location	 Canada Toronto Akamai Technologies Inc.
ASN	 AS16625 AKAMAI-AS - Akamai Technologies, Inc., US (registered May 30, 2000)
Resolve Host	a184-84-32-184.deploy.static.akamaitechnologies.com
Whois Server	whois.arin.net
IP Address	184.84.32.184

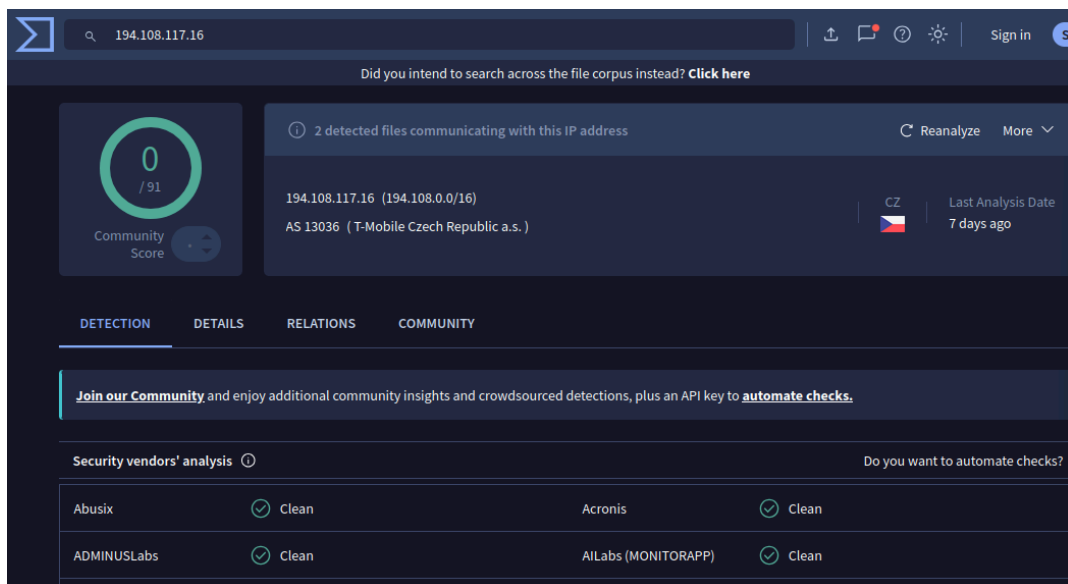
Picture 7. WHOIS lookup for 184.84.32.184

This resolves to AS16625, Akamai Technologies — a major CDN provider. CDN infrastructure hosting high volumes of ordinary traffic is expected and common. However, it is too early to conclude.

Reputation checks on both IPs came back clean from VirusTotal.

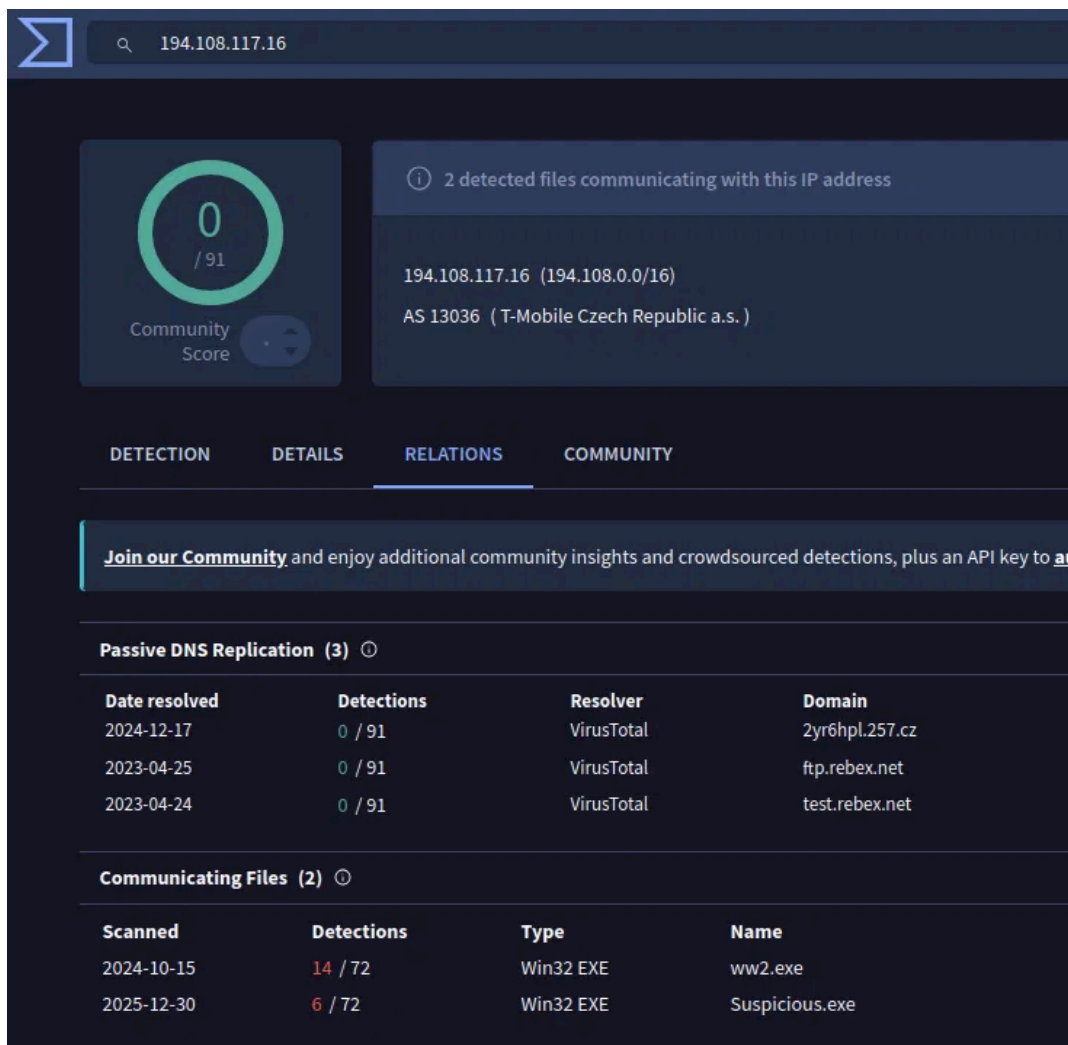


Picture 8. VirusTotal IP reputation lookup for 184.84.32.184



Picture 9. VirusTotal IP reputation lookup for 194.108.117.16

VirusTotal reports 8 detected files communicating with 184.84.32.184, and 2 detected files for 194.108.117.16. This means malicious files have previously been observed communicating with both addresses, even though the IPs themselves aren't flagged. This keeps both hosts open to investigation. I have not yet opened the file-relations tab for either IP to see what those files were — that's the next useful step, particularly for the T-Mobile FTP host given the earlier residential-hosting anomaly. I opened the Relations tab for 194.108.117.16 to see the file-association data referenced in the previous step.

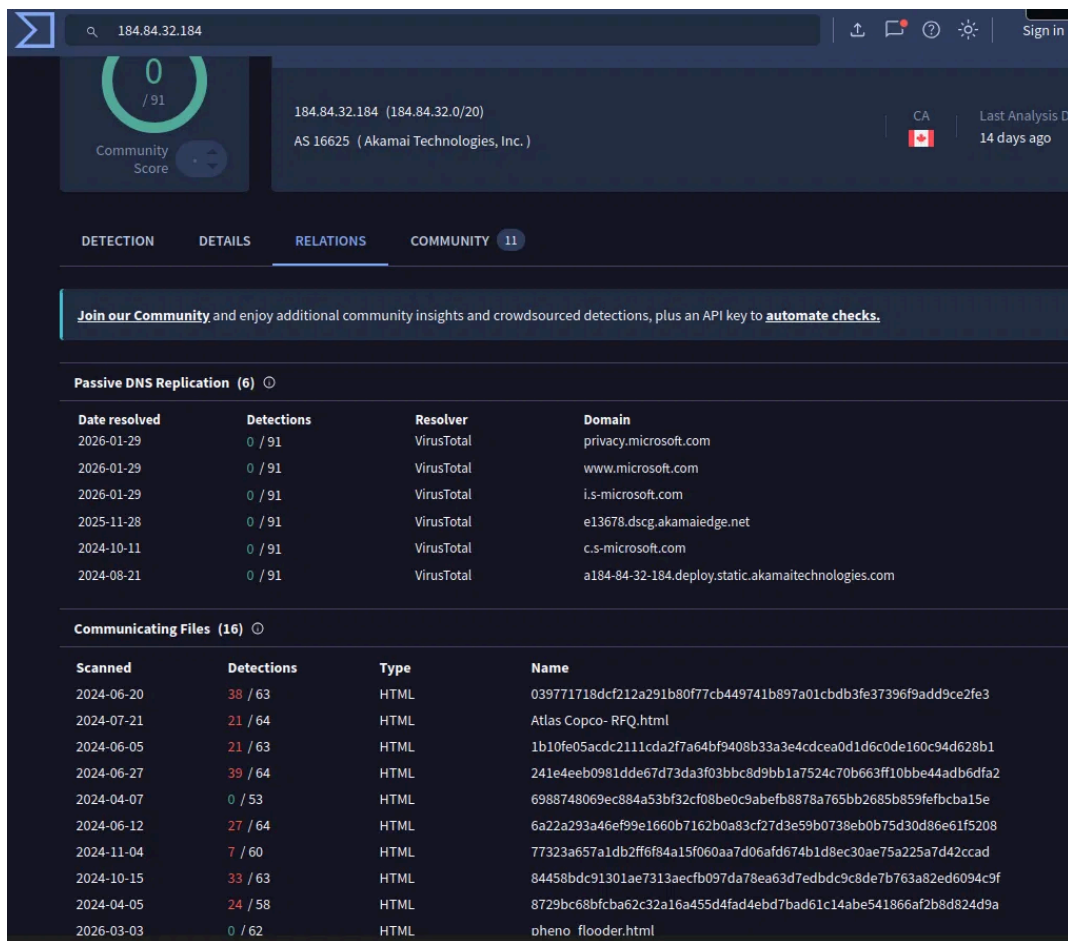


Picture 10. VirusTotal Relations tab for 194.108.117.16

Passive DNS shows this IP has historically resolved to ftp.rebex.net and test.rebex.net. these are well-known public FTP test servers operated by Rebex, a company that provides FTP client/server components and hosts these specifically for developers to test FTP connections against.

Communicating Files shows two Win32 executables previously observed communicating with this IP. These are actual malware samples.

I opened the Relations tab for 184.84.32.184 as well.



Picture 11. VirusTotal Relations tab for 184.84.32.184

Communicating Files shows 16 associated files, several with notably high detection counts which read as phishing-kit landing pages rather than malware binaries.

I ran directional packet counts on the Akamai conversation to resolve the earlier ambiguity about which side was source vs. destination and the reverse direction.

```

4 13.107.42.14
4 184.244.42.193
mockingcyber@SOC201: ~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap -n tcp and src 10.0.2.10 and dst 184.84.32.184 --count
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
114 packets
mockingcyber@SOC201: ~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap -n tcp and dst 10.0.2.10 and src 184.84.32.184 --count
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
113 packets
mockingcyber@SOC201: ~/Desktop/Investigations/Network-Analysis$

```

Picture 12. Directional packet counts for the 10.0.2.10 - 184.84.32.184 conversation

114 packets outbound and 113 inbound is a near-even split. This is consistent with a normal bidirectional request and response session like HTTPS handshake. This roughly-even count, combined with the legitimate Microsoft/Akamai passive DNS pattern from the previous step, further supports treating this conversation as likely benign CDN traffic.

```

mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap -
n tcp and src 10.0.2.10 and dst 194.108.117.16 --count
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
71 packets
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap -
n tcp and dst 10.0.2.10 and src 194.108.117.16 --count
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
60 packets
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$

```

Picture 13. Directional packet counts for the 10.0.2.10 - 194.108.117.16 conversation

71 packets were outbound and 60 packets inbound. This is mildly more outbound. I began searching through the PCAP for any pointers to malicious activity using grep.

```

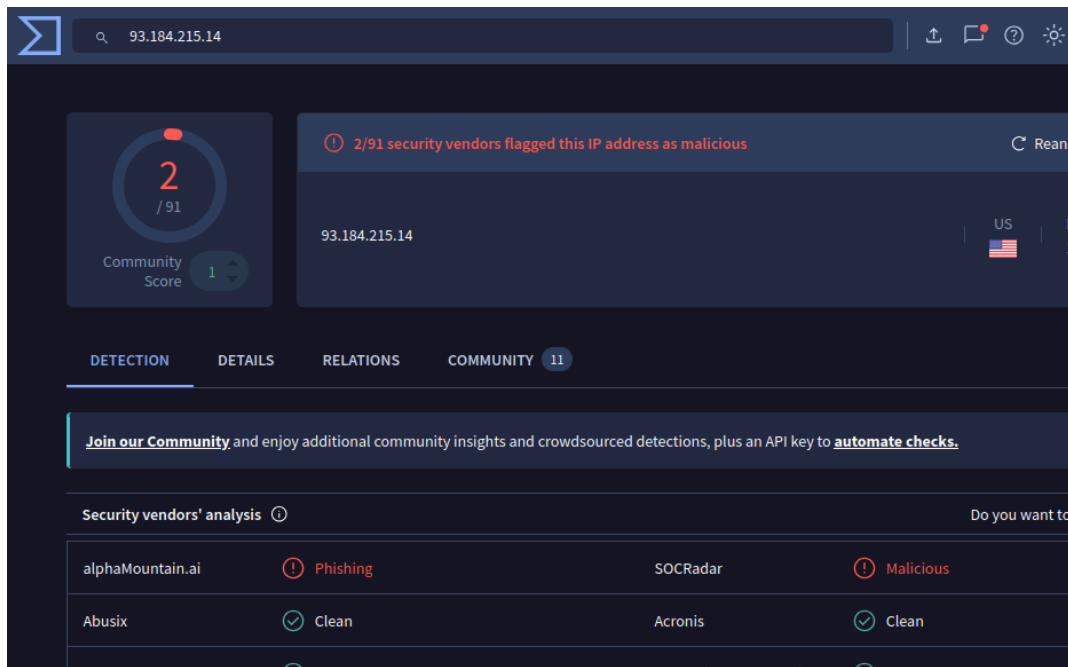
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap src 10.0.2.10 and dst 172.67.72.15 | grep -E "GET|POST"
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap dst 10.0.2.10 and src 172.67.72.15 | grep -E "GET|POST"
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap src 10.0.2.10 and dst 172.67.72.15 | grep "POST"
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap dst 10.0.2.10 and src 172.67.72.15 | grep "POST"
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap | grep "POST"
1716994492.929117 IP 10.0.2.10.34726 > 93.184.215.14.http: Flags [P.], seq 1:304, ack 1, win 32120, length 303: HTTP: POST / HTTP/1.1

```

Picture 14. Grep for HTTP GET/POST methods, narrowed then broadened searches

The narrowed searches against 172.67.72.15 returned no GET or POST matches in either direction. The broadened full-capture search surfaces a new, previously unflagged host with an HTTP POST from 10.0.2.10 to 93.184.215.14 on port 80 with 303 bytes of payload.

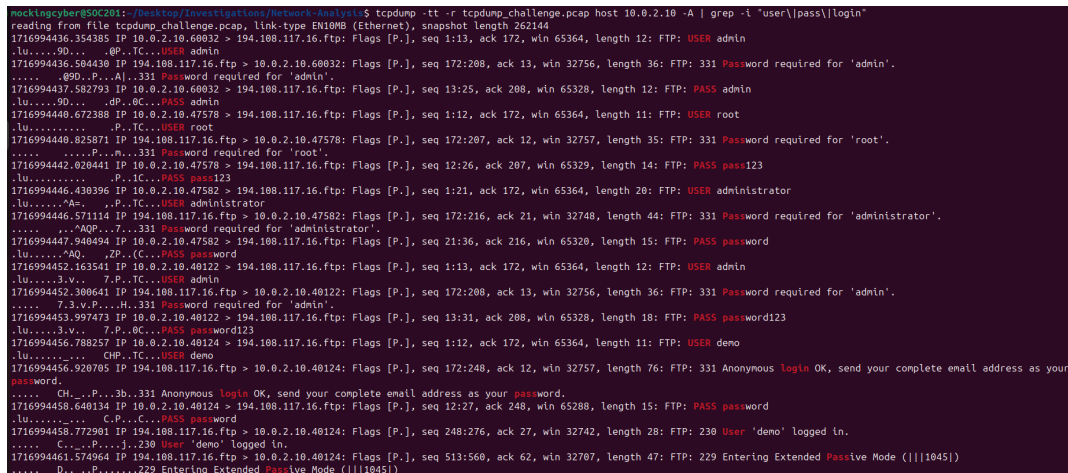
This is a new IP that hasn't appeared in any of the earlier top-talker or WHOIS/VirusTotal steps because it wasn't in the top of the full-IP frequency tally, meaning it's low-volume but was missed by a frequency-based approach. Furthermore, an outbound POST with a real payload is the first concrete evidence in this investigation of the endpoint actually *sending data* to an external host over HTTP. This is directly relevant to the info-stealer hypothesis. I ran 93.184.215.14 through VirusTotal.



Picture 15. VirusTotal reputation lookup for 93.184.215.14

2/91 vendors flag this address as malicious. This is the first host in the investigation with an active malicious flag on the IP itself. An actual outbound POST with a 303-byte payload to this address combined with the IP reputation is troubling.

I searched the capture for FTP credential activity



Picture 16. Credential activity search using grep

10.0.2.10 made five sequential login attempts against 194.108.117.16, each on a different source port, indicating separate connections.

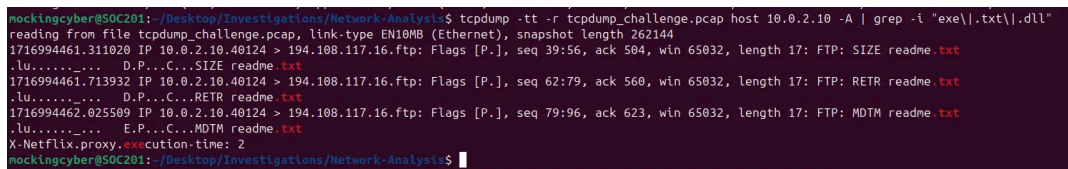
- USER: admin / PASSWORD: admin
- USER: root / PASSWORD: pass123

- USER: administrator / PASSWORD: password
- USER: admin / PASSWORD: password123
- USER: demo / PASSWORD: password (Successful)

This is a credential-guessing pattern against common default username-password combinations. This is a textbook automated brute-forcing. The server responded to one attempt with "Anonymous login OK, send your complete email address as your password". This is a standard FTP hint, however, the server never acted on it, since the client kept trying named accounts instead.

More significantly, the direction matters. 10.0.2.10 is the one initiating the brute-force attempt. This isn't the endpoint being attacked, but most likely the compromised endpoint's malware actively attacking an external FTP server, consistent with info-stealer/dropper behavior that automatically hunts for a writable drop server to exfiltrate stolen data to.

The successful login (demo/password) is immediately followed by "229 Entering Extended Passive Mode". This is the server opening a passive-mode data channel on port 1045 for a file transfer. I proceeded to search for file operation commands or any files present in the capture.



```
hackingcyber@SOC201: ~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap host 10.0.2.10 -A | grep -i "exe|.txt|.dll"
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
1716994461.311020 IP 10.0.2.10.40124 > 194.108.117.16.ftp: Flags [P.], seq 39:56, ack 504, win 65032, length 17: FTP: SIZE readme.txt
.lu..... D.P...C...SIZE readme.txt
1716994461.713932 IP 10.0.2.10.40124 > 194.108.117.16.ftp: Flags [P.], seq 62:79, ack 560, win 65032, length 17: FTP: RETR readme.txt
.lu..... D.P...C...RETR readme.txt
1716994462.025509 IP 10.0.2.10.40124 > 194.108.117.16.ftp: Flags [P.], seq 79:96, ack 623, win 65032, length 17: FTP: MDTM readme.txt
.lu..... E.P...C...MDTM readme.txt
X-Netflix.proxy.exe:cution-time: 2
hackingcyber@SOC201: ~/Desktop/Investigations/Network-Analysis$
```

Picture 17. FTP file operation commands following the successful login

RETR is the FTP download command which means the client is pulling readme.txt from the server. Combined with SIZE and MDTM, this looks like deliberate, scripted retrieval of a specific file. This is consistent with a malware stager or downloader pulling a second-stage payload or configuration file.

I searched for HTTP User-Agent strings across the host's traffic to attempt to locate any utilities that stand out.

```

mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap host 10.0.2.10 -A | grep -i "user-agent"
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: TeslaBrowser/5.5
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
class="c-heading-3">Your current User-Agent string appears to be from an automated process, if this is incorrect, p
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/535.1 (KHTML, like Gecko) Ubuntu/11.04 Chromium/14.0.825.0 Chrome/14.0.825.0 Safari/535.1

```

Picture 18. User-Agent strings across the capture

The user-agent TeslaBrowser/5.5 is very distinct. I looked it up and found some important information on Malpedia.

The screenshot shows the Malpedia website interface. At the top is the 'malpedia' logo and the 'Fraunhofer FKIE' logo. Below the logo is a navigation bar with links: 'Inventory', 'Statistics', 'Usage', 'ApiVector', and 'Login'. A search bar with the placeholder 'Quicksearch...' is visible. The search results show 'win.lumma (Back to overview)' and 'Lumma Stealer'. Below this, it says 'aka: LummaC2 Stealer' and 'Actor(s): Angry Likho'. There is a 'Propose Change' button. A small icon of a person is shown. Below the icon is a 'VTCollection' link. The main text describes Lumma Stealer as an information stealer written in C language, available through a Malware-as-a-Service (MaaS) model on Russian-speaking forums since at least August 2022. It is believed to have been developed by the threat actor 'Shamel', who goes by the alias 'Lumma'. Lumma Stealer primarily targets cryptocurrency wallets and two-factor authentication (2FA) browser extensions, before ultimately stealing sensitive information from the victim's machine. Once the targeted data is obtained, it is exfiltrated to a C2 server via HTTP POST requests using the user agent 'TeslaBrowser/5.5'. The stealer also features a non-resident loader that is capable of delivering additional payloads via EXE, DLL, and PowerShell.

Picture 19. Malpedia results for TeslaBrowser/5.5

This is a direct, credible match. Malpedia's entry states Lumma Stealer "exfiltrated to a C2 server via HTTP POST requests using the user agent 'TeslaBrowser/5.5.'" That's an exact string match to the user-agent flagged in the previous step, from a specialized malware-reference source.

Lumma Stealer is described as an info-stealer that "primarily targets cryptocurrency wallets and 2FA browser extensions" and exfiltrates via **HTTP POST**, not FTP. This lines up with the POST request to 93.184.215.14 as the actual exfiltration event. The FTP RETR of readme.txt look like a separate, secondary behavior. This could point to

the Malpedia note that Lumma features a non-resident loader capable of delivering additional payloads via EXE, DLL, and PowerShell. The FTP retrieval may be this loader fetching a component.

I re-ran the TeslaBrowser/5.5 search with extended context

```
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$ tcpdump -tt -r tcpdump_challenge.pcap host 10.0.2.10 -A | grep TeslaBrowser/5.5 -A 20
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
User-Agent: TeslaBrowser/5.5
Accept: */*

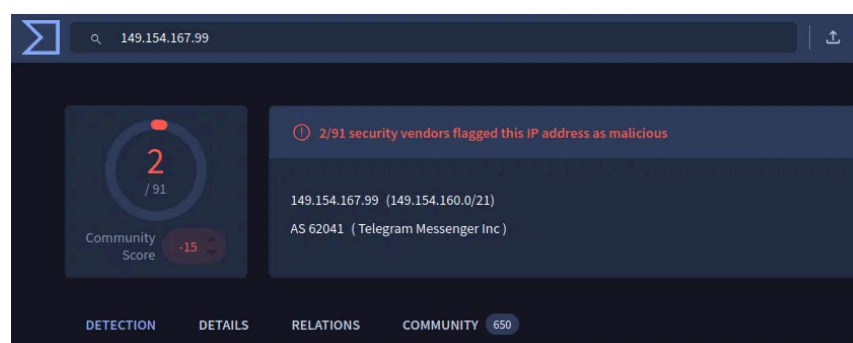
1716994467.338084 IP 149.154.167.99.http > 10.0.2.10.60086: Flags [P.], seq 1:371, ack 87, win 32682, length 370: HTTP: HTTP/1.1 301 Moved Permanently
E.....c
..
.P... ->Y..P.....HTTP/1.1 301 Moved Permanently
Server: nginx/1.18.0
Date: Wed, 29 May 2024 14:55:00 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://t.me/+zz01921skaaa

<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx/1.18.0</center>
</body>
mockingcyber@SOC201:~/Desktop/Investigations/Network-Analysis$
```

Picture 20. TeslaBrowser/5.5 request and server response, full context

This gives the fullest picture yet of the exfiltration channel. A request bearing the TeslaBrowser/5.5 user-agent receives a **301 Moved Permanently** response from 149.154.167.99, served by nginx/1.18.0, redirecting to a Telegram invite link <https://t.me/+zz01921skaaa>.

Using a Telegram channel as C2 infrastructure is a known technique for commodity info-stealers. Telegram's infrastructure is legitimate and rarely blocked outbound, which makes it a resilient redirector that's hard to block without collateral damage. The initial connection routed through 149.154.167.99, Telegram's own IP space, before landing on the invite link. This intermediate hop is likely a failsafe. If one drop point gets taken down, the operator updates the Telegram channel instead of the malware's hardcoded infrastructure. I ran 149.154.167.99 through VirusTotal to confirm reputation.



Picture 21. VirusTotal reputation lookup for 149.154.167.99

The VirusTotal results matches the expected Telegram infrastructure range. 2/91 vendors flag it malicious, with a notably low Community Score of -15, and a comparatively high engagement count. This is consistent with a legitimate-but-abused platform. The IP itself belongs to Telegram, but a meaningful body of community-submitted reports treats it as risky, which lines up with using Telegram as a C2 redirector rather than the platform itself being malicious infrastructure.

Verdict

The traffic in tcpdump_challenge.pcap confirms the endpoint is compromised with what is very likely Lumma Stealer, an info-stealer malware family. The identification rests on a direct, source-confirmed match. The User-Agent string "TeslaBrowser/5.5" observed in the capture is documented by Malpedia as Lumma Stealer's specific exfiltration signature. Traffic bearing this user-agent was redirected via a 301 response through Telegram infrastructure at 149.154.167.99 to a Telegram invite link, which is consistent with Lumma's known use of resilient, hard to block C2 redirection.

Separately, the endpoint conducted automated credential brute forcing against an external FTP server at 194.108.117.16 and used a successful login to retrieve a file named readme.txt via RETR. This behavior is consistent with Lumma's documented non-resident loader fetching a secondary component, and it appears distinct from the core stealer's exfiltration path. An outbound POST to 93.184.215.14, an IP independently flagged for phishing, represents a second, separately confirmed suspicious point, although its exact relationship to the Lumma infrastructure has not been established from this capture alone.

Traffic at 172.67.72.15 was ICMP only, and the traffic at 184.84.32.184 was also reviewed. Both were assessed as unrelated, benign background traffic.

MITRE ATT&CK Mapping

- Command and Control — T1071.001 (Application Layer Protocol: Web Protocols): HTTP-based C2 traffic to attacker infrastructure.
- Command and Control — T1102 (Web Service): Use of Telegram as a C2 redirector/dead-drop resolver.
- Credential Access — T1110.001 (Brute Force: Password Guessing): Automated sequential login attempts against the FTP server.
- Exfiltration — T1041 (Exfiltration Over C2 Channel): Consistent with Lumma's documented HTTP POST exfiltration method.

Indicators of Compromise (IOCs)

Affected Host: 10.0.2.10

URLs: t[.]me/+zz01921skaaa

IPs:

- 149.154.167.99
- 93.184.215.14
- 194.108.117.16 (FTP brute-force target, secondary/loader activity)

User-Agent: TeslaBrowser/5.5

File: readme.txt

Credentials: USER - demo, PASS - password (ftp)

Recommendations

1. Isolate infected host from the network immediately to stop further C2 communication and prevent lateral movement.
2. Block outbound traffic to 149.154.167.99, 93.184.215.14, and 194.108.117.16 at the firewall/proxy level.
3. Retrieve and analyze readme.txt from the FTP session.
4. Treat any credentials stored in the browser or OS on this endpoint as compromised, given Lumma's documented targeting of browser-stored credentials and 2FA extensions. Force password resets for the affected user.
5. Check whether TeslaBrowser/5.5 or the same C2 indicators appear on other endpoints to determine if this is isolated or part of a wider compromise.
6. Preserve the FTP server credentials used (194.108.117.16) as a secondary IOC. The successful demo/password login may indicate a known-weak or default-configured drop server used across multiple Lumma infections.